

Trabalho Prático – Módulo 1

André Graça
Daniel Caseiro
Diogo Santos

Orientadores Vítor Fialho
 Artur Ferreira

Relatório realizado no âmbito de Comunicação Digital,
do curso de licenciatura em Engenharia Informática e de Computadores
Semestre de Verão 2022/23

Maio de 2023

Instituto Superior de Engenharia de Lisboa

Licenciatura em Engenharia Informática e de Computadores

Trabalho Prático – Módulo 1

47224 André Graça

46052 Daniel Caseiro

48459 Diogo Santos

Orientadores: Vítor Fialho
Artur Ferreira

Relatório realizado no âmbito de Comunicação Digital,
do curso de licenciatura em Engenharia Informática e de Computadores
Semestre de Verão 2022/23

Maio de 2023

Resumo

Este relatório apresenta os resultados de um conjunto de exercícios que abordam temas relacionados com a teoria da informação e comunicação. O trabalho envolve a implementação de funções e algoritmos em linguagem Python e C, a análise de fontes de símbolos, a implementação de fontes de símbolos e a aplicação de técnicas de criptografia e comunicação de dados. Os resultados obtidos foram avaliados com base em medidas de desempenho, tais como a entropia e a taxa de erro de bit, entre outras. O trabalho permitiu aprofundar conhecimentos sobre os conceitos fundamentais da teoria da informação e comunicação, bem como as suas aplicações práticas em diversos domínios. O relatório apresenta uma síntese dos principais resultados obtidos e as principais conclusões do trabalho realizado.

Palavras-chave: teoria da informação; comunicação; implementação de funções; algoritmos; Python; linguagem C; análise de fontes de símbolos; fontes de símbolos; técnicas de criptografia.

Abstract

This report presents the results of a set of exercises that address topics related to information and communication theory. The work involves the implementation of functions and algorithms in Python and C languages, the analysis of symbol sources, the implementation of symbol sources, and the application of cryptography and data communication techniques. The results obtained were evaluated based on performance measures, such as entropy and bit error rate, among others. The work allowed deepening knowledge about the fundamental concepts of information and communication theory, as well as their practical applications in various domains. The report presents a summary of the main results obtained and the main conclusions of the work carried out.

Keywords: information theory; communication; implementation of functions; algorithms; Python; C language; analysis of symbol sources; symbol sources; cryptography techniques.

Índice

RESUMO	IV
ABSTRACT	VI
LISTA DE FIGURAS	X
1. INTRODUÇÃO	1
1.3 ORGANIZAÇÃO DO DOCUMENTO	2
2. FORMULAÇÃO DO PROBLEMA	4
2.1 AULA PRÁTICA 1	4
2.2 TRABALHO PRÁTICO 1	5
3. SOLUÇÃO PROPOSTA - GRANDES IDEIAS	8
3.1 ALINEA 1	8
3.1.1.1 (4) Alínea (a) – count_ones.c	8
3.1.1.2 (4) Alínea (b) – print_bits.c	9
3.1.1.3 (4) Alínea (c) – most_frequent_symbols.c	9
3.1.1.4 (4) Alínea (d) – negative_file.c	9
3.1.1.5 (5) Alínea (a) – geometric_prog.py	10
3.1.1.6 (5) Alínea (b) – Euclides_algorithm.py	10
3.1.1.7 (5) Alínea (c) – most_and_least_frequent_symbols.py	11
3.1.1.8 (5) Alínea (d) – histogram.py	11
3.1.2 Alínea 2	18
3.1.2.1 Alínea (a) – histogram.py	18
3.1.3 Alínea 3	18
3.1.3.1 Alínea (a) - GenericSource.py	18
3.1.3.2 Alínea (b) - SequenceGenerator.py	19
3.1.3.2 Alínea (c) – PasswordGenerator.py	19
3.1.4 Alínea 4	20
3.1.4.1 Alínea (a) – VernamCypher.py	20
3.1.4.2 Alínea (b) – AliceCypher.py	20
3.1.5 Alínea 5	21
3.1.5.1 Alínea (a) – Binary Symmetric Channel	21
3.2 EXPRESSÕES MATEMÁTICAS	23
4. AVALIAÇÃO EXPERIMENTAL	25
4.1 ANÁLISE EXPERIMENTAL	25

4.1.2 Alínea 5	25
4.1.2.1 (a) - geometric_progression_test.py	25
4.1.2.2 (b) - EuclidesAlgorithm_test.py	26
4.1.2.3 (c) - most_and_least_frequent_symbols.py.....	26
4.1.2.4 (d) - histogram.py	27
4.3 TESTES AUXILIARES	ERROR! BOOKMARK NOT DEFINED.
5. CONCLUSÕES.....	29
REFERÊNCIAS	30

Lista de Figuras

Figura 1 - Histograma da alice29.txt.....	13
Figura 2 - Entropia da alice29.txt.....	13
Figura 3 - Histograma do lena.bmp.....	14
Figura 4 - Entropia do lena.bmp.....	14
figura 5 - Histograma e Entropia do a.txt.....	15
figura 6 - Histograma e Entropia do cp.htm.....	15
figura 7 - Histograma e Entropia do Person.java	16
figura 8 - Histograma e Entropia do Progc.c.....	16
figura 9 - Histograma e Entropia do ListaPalavrasPT.txt	17
figura 10 - Histograma e Entropia do ListaPalavrasEN.txt.....	17

1. Introdução

O presente relatório descreve o trabalho realizado no âmbito da disciplina de Comunicações Digitais, com o objetivo de explorar e analisar diferentes conceitos e técnicas relacionadas a esse campo de estudo. O trabalho consiste na análise e implementação de funções, fontes de símbolos, cifra de Vernam para ficheiros de texto e implementação do modelo de canal binário simétrico e técnicas de interleaving.

Na primeira parte do trabalho, foram realizados exercícios práticos referentes aos conceitos básicos de comunicações digitais que nos foram pedidos. Foram implementadas funções e programas de teste correspondentes, os quais foram utilizados para comprovar o funcionamento das mesmas por meio de resultados experimentais.

Em seguida, foram analisadas fontes de símbolos, tanto por meio de arquivos de teste de texto e de imagem. Foram calculados histogramas, valores de informação própria de cada símbolo e entropias dos arquivos, permitindo uma compreensão mais aprofundada sobre as características das fontes de símbolos analisadas.

Foi realizada a implementação de fontes de símbolos genéricas, permitindo a geração de sequências de símbolos com base na função massa de probabilidade do alfabeto de símbolos. Essas sequências foram geradas para diferentes dimensões e comparadas com o valor estimado da entropia da fonte, proporcionando uma análise sobre a relação entre os valores M e N na entropia resultante.

Posteriormente, foi abordada a cifra de Vernam para ficheiros de texto, com a implementação de uma função de cifragem e decifragem utilizando a operação XOR bit a bit. Foram realizadas cifras e decifras de uma sequência específica, bem como a cifragem de um arquivo de texto completo, comparando os histogramas e entropias do texto original e do texto cifrado.

Por fim, foi implementado o modelo de canal binário simétrico e técnicas de interleaving, incluindo a escrita de uma função que implementa o modelo de canal binário simétrico e a implementação da técnica de entrelaçamento de transmissão. Foram realizadas comparações e análises dos valores de BER (Bit Error Rate) em diferentes situações e configurações de entrelaçamento.

O relatório apresentará detalhes sobre as implementações realizadas, os resultados obtidos, as análises feitas e as conclusões derivadas de cada etapa do trabalho. Essas informações fornecerão uma compreensão mais sólida sobre os conceitos de Comunicações Digitais abordados, bem como sua aplicação prática.

1.3 Organização do documento

A organização de um documento é um dos aspectos mais importantes para garantir que ele é facilmente compreendido e valorizado pelos seus leitores. Ao seguir a estrutura de Capa, Índice, Resumo, Abstract, Lista de Figuras, Lista de Tabelas, Introdução, Formulação do Problema, Solução Proposta, Avaliação Experimental, Conclusões e Referências, é possível garantir que todas as informações importantes estão incluídas e apresentadas de uma forma lógica e coerente.

2. Formulação do Problema

2.1 Aula Prática 1

A fim de abordar adequadamente as questões apresentadas em cada alínea, é necessário aplicar o conhecimento adquirido em C durante o curso de PSC e familiarizar-se com a linguagem Python, conforme exigido.

Em particular, devemos levar em consideração o tamanho em bits de tipos de dados, como o inteiro (32 bits), e ter uma compreensão básica das operações lógicas, como a negação (NOT) e algumas fórmulas matemáticas como as de progressão geométrica. Esses conceitos serão essenciais para implementar corretamente as funções propostas e obter os resultados desejados.

2.1.1 Linguagem C

(a) Para abordar o problema de contar o número de bits iguais a '1' e '0' em um valor inteiro, eu implementaria as funções `count_ones` e `count_zeros` utilizando uma máscara binária que é deslocada bit a bit sobre o valor. A cada iteração, eu verificaria se o bit correspondente é igual a '1' ou '0' e incrementaria o contador apropriado.

(b) Para imprimir os valores dos bits de um valor inteiro como caracteres, eu utilizaria a função `print_bits` para percorrer cada bit do valor. A cada iteração, eu converteria o bit em um caractere correspondente ('1' ou '0') e imprimiria na tela.

(c) Para encontrar o símbolo mais frequente em um arquivo de texto, eu implementaria a função `most_frequent_symbol` lendo o ficheiro e incrementando um contador correspondente a esse símbolo, mantendo sempre o símbolo mais aparecido numa variável, no fim bastava retorná-la.

(d) Para transformar um arquivo de entrada invertendo cada bit, a função `negative_file` leria o arquivo de entrada e, para cada byte lido, negaria cada bit individualmente. Em seguida, os bytes modificados seriam gravados no arquivo de saída, resultando em uma representação invertida do arquivo original.

2.1.2 Linguagem Python

(a) No caso da função que apresenta os primeiros N termos de uma progressão geométrica, será necessário implementar um loop que calcule os termos da sequência com base no primeiro termo (u) e na razão (r) fornecidos como parâmetros.

(b) Para determinar o máximo divisor comum entre dois números inteiros usando o algoritmo de Euclides, será necessário implementar uma função que realize repetidamente as divisões necessárias até encontrar o resultado.

(c) A função que identifica os símbolos mais e menos frequentes de um arquivo exigirá a leitura do arquivo e a contagem das ocorrências de cada símbolo. Ao percorrer o arquivo, será necessário manter um registro dos símbolos mais e menos frequentes, bem como suas respectivas frequências.

(d) Para apresentar o histograma de um arquivo, calcular a informação própria de cada símbolo e a entropia do arquivo, será necessário realizar a leitura do arquivo e processar os símbolos encontrados. Ao contar as ocorrências de cada símbolo, poderemos calcular a frequência relativa e, conseqüentemente, a informação própria. A partir desses valores, poderemos calcular a entropia do arquivo.

2.2 Trabalho Prático 1

A formulação do problema é um dos passos mais importantes na resolução de qualquer desafio, pois permite identificar e compreender os obstáculos que precisam de ser ultrapassados. No contexto do nosso relatório, vamos explorar diferentes problemas relacionados com a análise e implementação de fontes de símbolos, bem como com a cifra de Vernam e a técnica de entrelaçamento de transmissão.

Em relação à análise de fontes de símbolos, vamos examinar a forma como os dados são distribuídos e como é possível medir a informação e entropia de um conjunto de símbolos. Iremos também comparar os resultados obtidos em diferentes conjuntos de dados, nomeadamente em ficheiros de texto em língua inglesa e portuguesa.

No que diz respeito à implementação de fontes de símbolos, iremos focar-nos na criação de sequências com diferentes dimensões e no estudo da sua entropia. Vamos também explorar a

utilização destas sequências na geração de palavras-passe robustas, bem como a comparação dos resultados obtidos com os valores esperados.

No âmbito da cifra de Vernam, vamos analisar a sua aplicação em ficheiros de texto, nomeadamente a sua implementação e os resultados obtidos através da cifragem e decifragem de uma sequência de caracteres. Vamos também comparar os histogramas e entropias do texto em claro e do texto cifrado, bem como examinar o seu desempenho com chaves constantes e aleatórias.

Por fim, iremos abordar a técnica de entrelaçamento de transmissão e a sua implementação num modelo de canal binário simétrico. Vamos explorar a sua utilização na mitigação de erros de transmissão, bem como a comparação dos resultados obtidos com diferentes dimensões de matriz de interleaving e valores de BER.

3. Solução Proposta - Grandes Ideias

Neste conjunto de exercícios, vamos abordar diversos tópicos relacionados com processamento de informação e comunicação de dados. A implementação funções para análise de fontes de símbolos, como histogramas e cálculo da entropia, como também implementar uma fonte de símbolos genérica e gerar sequências com diferentes dimensões, comparando o valor da entropia da fonte com o valor estimado para a entropia das sequências geradas.

Posteriormente, vamos implementar a cifra de Vernam para ficheiros de texto e realizar a cifra e decifra de uma sequência de caracteres. Em adição, realizar a cifra de um arquivo de texto inteiro com a chave constante e com uma chave aleatória e comparar os resultados usando histogramas e cálculo de entropia.

De seguida, a implementação o modelo de canal binário simétrico e técnicas de interleaving, incluindo também, a escrita de uma função que implemente o modelo de canal binário simétrico e comparar o valor do Bit Error Rate (BER) passado como parâmetro com o valor real obtido, tendo também em consideração implementar a técnica de entrelaçamento e realizar o sistema proposto para avaliar o seu funcionamento.

Ao longo destes exercícios, iremos trabalhar com diversos conceitos importantes em processamento de informação e comunicação de dados, tais como funções de probabilidade, cifras de segurança, canais de comunicação e técnicas de transmissão de dados.

3.1 Alinea 1

3.1.1.1 (4) Alínea (a) – *count_ones.c*

Esta função `count_ones` recebe um inteiro e retorna o número de bits iguais a 1 nesse valor.

A função `count_ones` utiliza um loop que é percorrido enquanto a máscara de bits percorrer cada bit do inteiro. A máscara começa com o valor 1 (representando o bit menos significativo) e é deslocada para a esquerda em cada iteração, permitindo verificar cada bit do valor `val`. Se o resultado da operação binária `val & mask` for diferente de zero, significa que o bit correspondente é igual a 1, e nesse caso, o contador `res` é incrementado.

3.1.1.2 (4) Alínea (b) – *print_bits.c*

Esta função imprime a representação binária de um valor inteiro como uma sequência de caracteres. Ela percorre, com uma máscara, os bits do parametro, começando pelo bit mais significativo, e imprime '1' se o bit for 1 ou '0' se o bit for 0.

3.1.1.3 (4) Alínea (c) – *most_frequent_symbols.c*

Esta função recebe um ponteiro para char, que representa o nome do ficheiro a analisar e retorna o simbolo que ocorre mais vezes nesse ficheiro. Para isso, ela começa por criar um array com 256 posições iniciadas com um contador a 0, para representar os possiveis 256 simbolos. Por cada simbolo que encontrarmos no ficheiro enquanto o percorremos, incrementamos a posição correspondente ao código ASCII dele, registamos na variável maxCount a contagem se esta for menor e também igualamos res ao simbolo atual, por fim retornamos res.

3.1.1.4 (4) Alínea (d) – *negative_file.c*

Este código em linguagem C implementa uma função chamada *negative_file* que transforma um arquivo de entrada em um arquivo de saída, aplicando a negação bit a bit em cada caractere do arquivo de entrada. A função `void negative_file(char *input_file_name, char *output_file_name)` recebe como entrada o nome do arquivo de entrada e o nome do arquivo de saída. O tipo de retorno da função é void, o que significa que ela não retorna nenhum valor.

De forma geral, a função entra em num loop que percorre cada caractere do arquivo de entrada. A instrução `while((t = getc(input_file)) != EOF)` lê o próximo caractere do arquivo de entrada e armazena na variável t. A condição do loop verifica se o valor retornado pela função `getc` é diferente do valor EOF, que indica o final do arquivo.

Dentro do loop, a instrução `fputc(t,output_file)` grava cada caractere no arquivo de saída. No entanto, antes de gravar o caractere, a instrução `t = ~t` é usada para aplicar a negação bit a bit no caractere. A operação `~` inverte todos os bits do número, transformando cada 0 em 1 e cada 1 em 0.

Após o loop terminar, os arquivos de entrada e saída são fechados usando as funções `fclose(input_file)` e `fclose(output_file)`, respectivamente.

Resumindo, o código implementa uma função em C que lê um arquivo de entrada, aplica a negação bit a bit em cada caractere e grava o resultado em um arquivo de saída. O código é relativamente simples, usando funções básicas de entrada e saída de arquivos em C.

3.1.1.5 (5) Alínea (a) – *geometric_prog.py*

O código em Python apresentado define uma função denominada "geometric_prog" que recebe três parâmetros: u, r e n. Essa função calcula os primeiros n termos da progressão geométrica com primeiro termo u e razão r.

Na primeira linha da função, é definida uma lista vazia denominada "res", que será usada para armazenar os resultados da progressão geométrica. Em seguida, é criada uma lista de números de 0 até n-1, denominada "numbers", que será usada para controlar o número de iterações do laço "for" a seguir.

O laço "for" percorre os números de 1 até n-1, ou seja, itera n-1 vezes. Para cada iteração, é calculado o valor do termo da progressão geométrica correspondente à iteração atual e adicionado à lista "res" usando o método "append()". O valor do termo é calculado usando a fórmula matemática $a(n) = u \cdot r^{(n-1)}$, onde n é o número da iteração atual.

Finalmente, a função retorna a lista "res" contendo os primeiros n termos da progressão geométrica. O código final usa a função "geometric_prog" para calcular e imprimir os primeiros 10 termos de uma progressão geométrica com primeiro termo 3 e razão 2. O resultado impresso é: [3, 6, 12, 24, 48, 96, 192, 384, 768].

3.1.1.6 (5) Alínea (b) – *Euclides_algorythm.py*

Este código apresenta a função MDC que recebe dois números inteiros D e d e retorna o máximo divisor comum entre eles, utilizando o algoritmo de Euclides.

O algoritmo de Euclides é um método eficiente para encontrar o máximo divisor comum entre dois números inteiros. O algoritmo baseia-se na observação de que, se o resto da divisão de D por d é zero, então d é o máximo divisor comum entre D e d. Caso contrário, o máximo divisor comum entre D e d é igual ao máximo divisor comum entre d e o resto da divisão de D por d.

Aqui está a explicação passo a passo do código:

1. A função MDC é definida com dois parâmetros D e d.
2. A variável r é inicializada com o resto da divisão de D por d.
3. Inicia-se um laço while com a condição $r \neq 0$, que será executado até que o resto da divisão de D por d seja zero.
4. Dentro do laço, a variável D recebe o valor de d, e a variável d recebe o valor de r.
5. A variável r é atualizada com o resto da divisão de D por d.
6. Quando a condição do laço while não é mais satisfeita (isto é, $r == 0$), o laço é interrompido e o valor de d é retornado como resultado da função.

7. No exemplo apresentado, a função é chamada com os argumentos 348 e 156, e o valor 12 é retornado como resultado. Isso significa que o máximo divisor comum entre 348 e 156 é 12.
8. A função print é usada para exibir a mensagem "Máximo divisor comum:" seguida do valor retornado pela função MDC.

3.1.1.7 (5) Alínea (c) – *most_and_least_frequent_symbols.py*

Este código é escrito em Python e tem como objetivo identificar os símbolos mais e menos frequentes em um arquivo passado como parâmetro.

A função `most_and_least_frequent_symbols(file)` recebe um nome de arquivo como parâmetro. Em seguida, ela abre o arquivo com o `open(file, 'r')`, lê o conteúdo do arquivo com o `read()`, remove as quebras de linha com o `replace("\n", "")`, e cria um objeto Counter com o conteúdo do arquivo, que irá contar a ocorrência de cada símbolo.

Os métodos `most_common(1)` e `most_common()[-1]` da classe Counter são utilizados para obter a tupla com o símbolo mais e menos comum, respectivamente. O primeiro elemento dessas tuplas é a letra mais ou menos comum e o segundo elemento é o número de ocorrências da letra. Essas tuplas são armazenadas nas variáveis `letterM` e `countM` (mais comum) e `letterL` e `countL` (menos comum).

Por fim, a função exibe na tela as informações obtidas com `print()`. A função `main()` chama a função `most_and_least_frequent_symbols()` com o primeiro argumento passado pelo terminal com `sys.argv[1]`.

3.1.1.8 (5) Alínea (d) – *histogram.py*

Este código consiste em uma função para gerar o histograma de um ficheiro, calcular a informação própria de cada símbolo e a entropia do ficheiro.

A função principal é `main()`, onde é possível definir o ficheiro que queremos analisar. Em seguida, é feita uma chamada à função `showResults()` passando como argumento o nome do ficheiro. Esta função irá chamar as funções necessárias para obter os resultados e apresentá-los na consola.

A primeira função que é chamada `readFile()` é usada para ler o conteúdo do ficheiro e colocá-lo numa *String*. Esta função recebe como parâmetro o nome do ficheiro. Esta função começa por

verificar se o tipo de ficheiro é BMP (Bit Map Picture), caso seja, é chamada a função `readbmp()` passando o nome do ficheiro como parâmetro e retornando um *array* de inteiros, que no exemplo da *lena.bmp*, estes inteiros seriam os tons de cinza, codificados desde 0 a 255. Caso seja outro tipo de ficheiro é retornado uma *String* com o conteúdo do mesmo.

A função `fmp()` é usada para com base no conteúdo do ficheiro gerar uma função de massa de probabilidade. Ela recebe como parâmetros uma *String* ou *array* de inteiros e retorna um Counter, um objeto cujo nome das suas propriedades são o caracter e o valor dessa propriedade é a probabilidade de ocorrência desse caracter. Esta função começa transformando a *String* em um Counter, em que o valor da sua propriedade é o número de ocorrências do caracter, em seguida é feito um ciclo para alterar o valor das propriedades do Counter para a percentagem de ocorrência com a seguinte fórmula: $\frac{\#ocorrências}{\#total\ de\ bits}$.

Em seguida é chamada a função `printPI()` que dado uma função massa de probabilidade apresenta o valor da *Informação Própria* de cada caracter. Esta função chama a função `personalInfo()` que dado a probabilidade de ocorrência do caracter calcula o valor da *Informação Própria* do caracter usando a fórmula $I(x) = -\log_2(p(x))$.

Em seguida é feita a chamada á função `entropy()` que dado uma função massa de probabilidade apresenta o valor da entropia do ficheiro. Esta função executa um ciclo para cada caracter chamando a função `personalInfo()` e multiplicando o resultado pela probabilidade de ocorrência do caracter, de forma a calcular a entropia $H(x) = -\sum p(x) * \log_2(p(x))$, depois apresenta o valor desta.

A função `main()` faz uma chamada á função `createHist()` que dado o nome do ficheiro, o N° de bins do histograma e a *String* com o conteúdo do ficheiro gera um histograma do ficheiro.

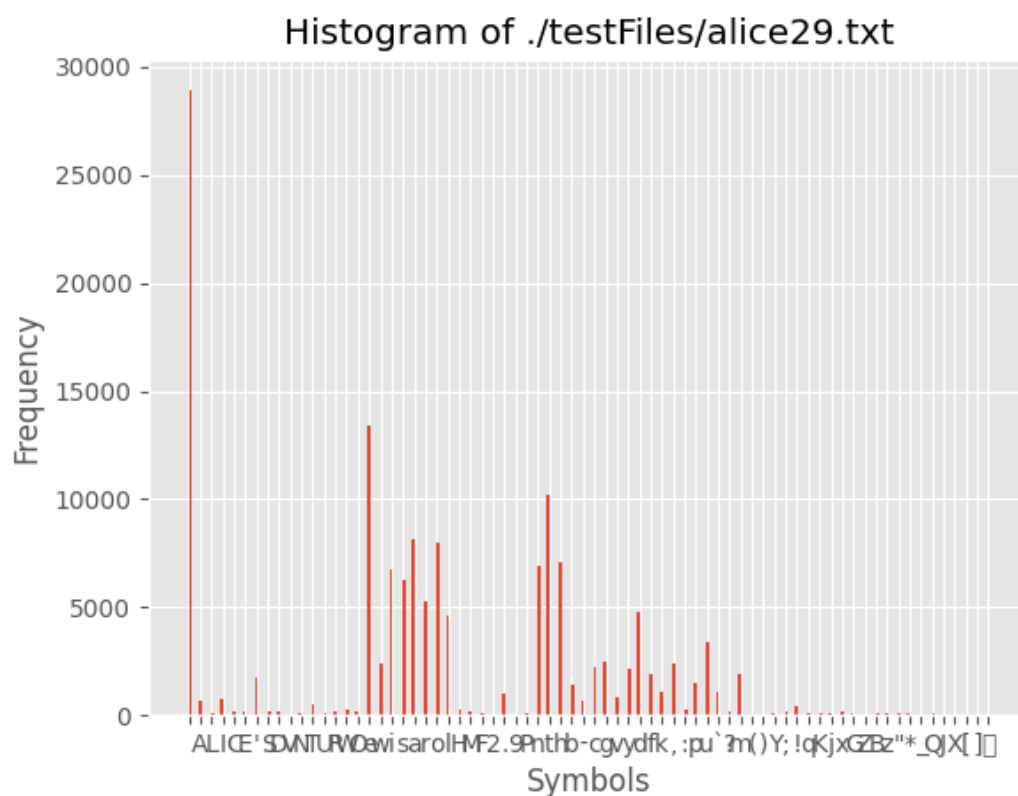


Figura 1 - Histograma da alice29.txt

Como podemos observar na figura acima, o caracter ' ' (espaço) é o caracter mais dominante no ficheiro *alice29.txt*, logo é expectável que a Informação Própria deste caracter seja a menor, visto que tem mais ocorrências é necessários menos bits para codificar.

Symbol : 2.325607385773178
Entropy is 4.45620220605495

Figura 2 - Entropia da alice29.txt

Na figura em cima está representado a Informação Própria do caracter ' ' (espaço) e a entropia do ficheiro, como podemos observar o valor da entropia é de 4.45, isto quer dizer que em média são necessários “4.45” bits para codificar um caracter do ficheiro *alice29.txt*.

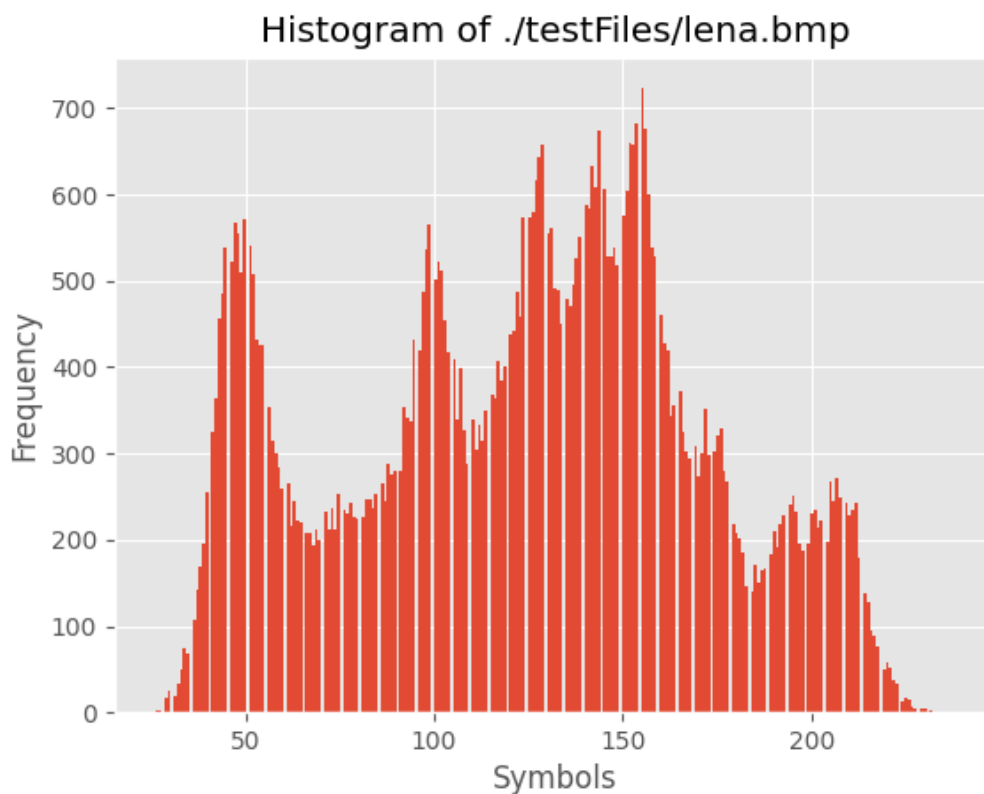


Figura 3 - Histograma do lena.bmp

Na figura acima está representada o histograma do ficheiro *lena.bmp*, neste histograma estão representados valores inteiros no eixo do 'x' em vez de caracteres, isto ocorre, pois, o ficheiro é uma imagem em que cada pixel tem um valor inteiro que corresponde a um tom de cinza.

Entropy is 7.424422621459047

Figura 4 - Entropia do lena.bmp

Na figura acima está representada a entropia do ficheiro *lena.bmp*, o valor 7.42 indica que em média o número de bits necessários para codificar um carácter é de “7.42” bit, isto quer dizer que alguns caracteres poderão ser codificados apenas com 7 ou 6 bits.

De seguida irão estar representados os restantes histogramas e as suas respetivas entropias.

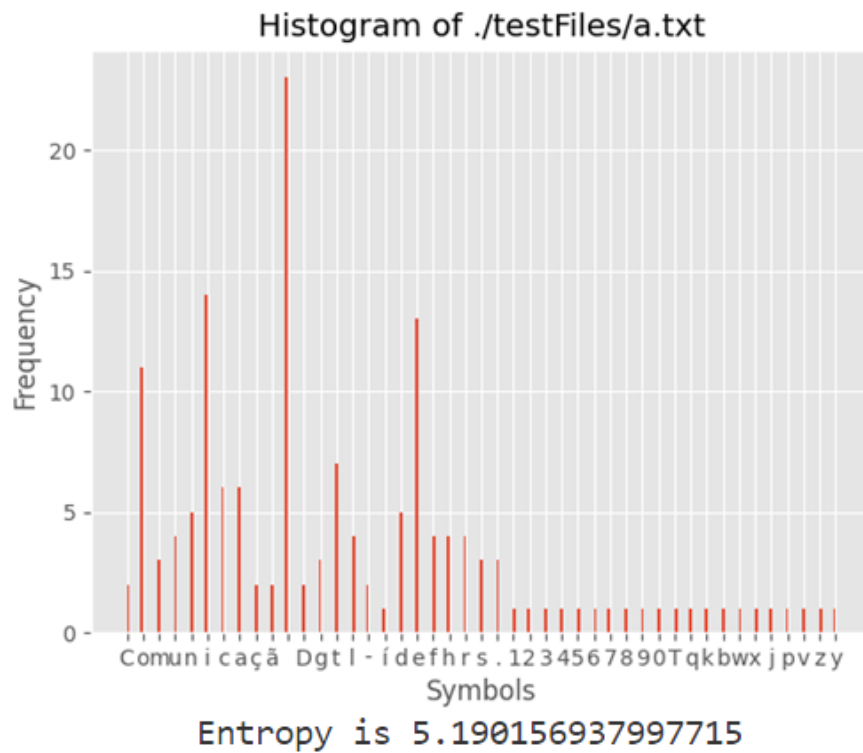


figura 5 - Histograma e Entropia do a.txt

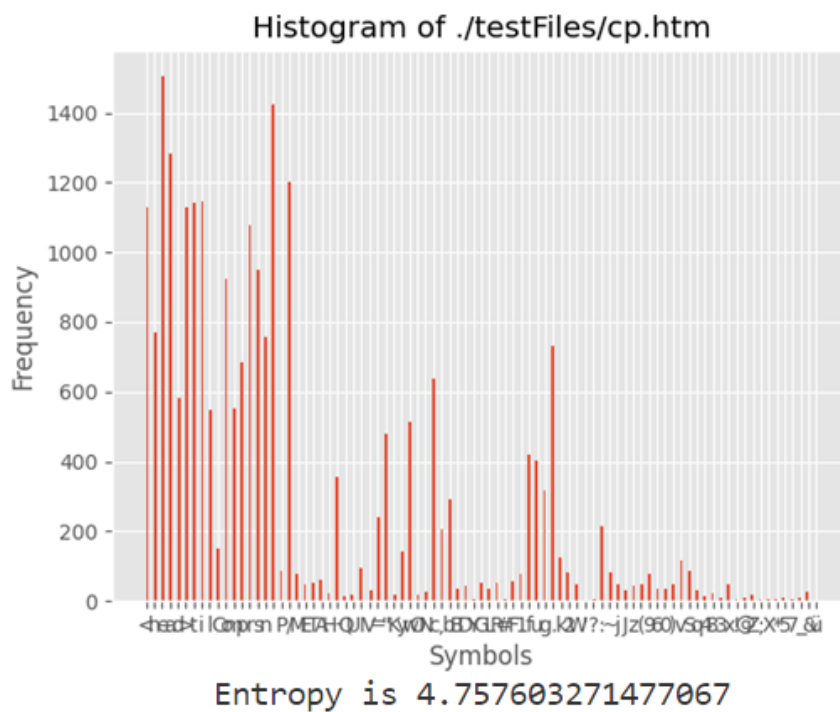


figura 6 - Histograma e Entropia do cp.htm

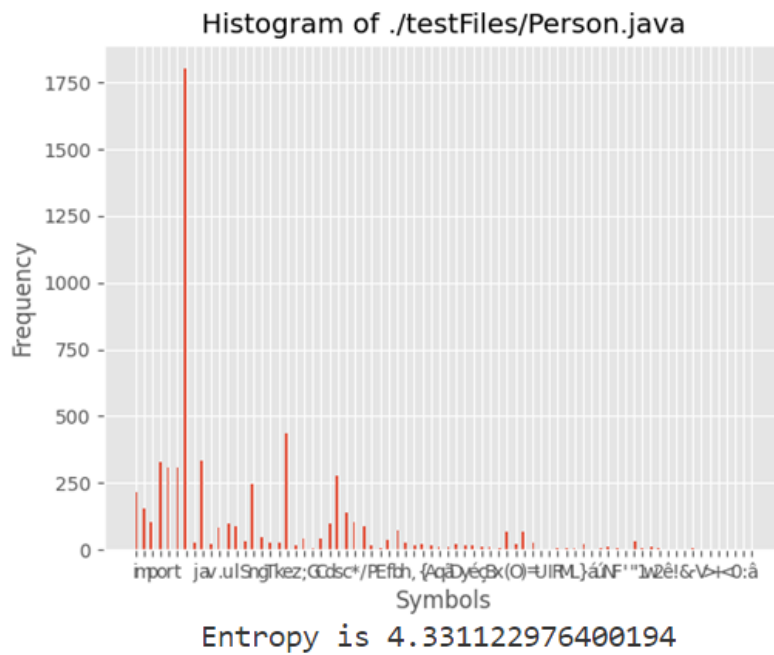


figura 7 - Histograma e Entropia do Person.java

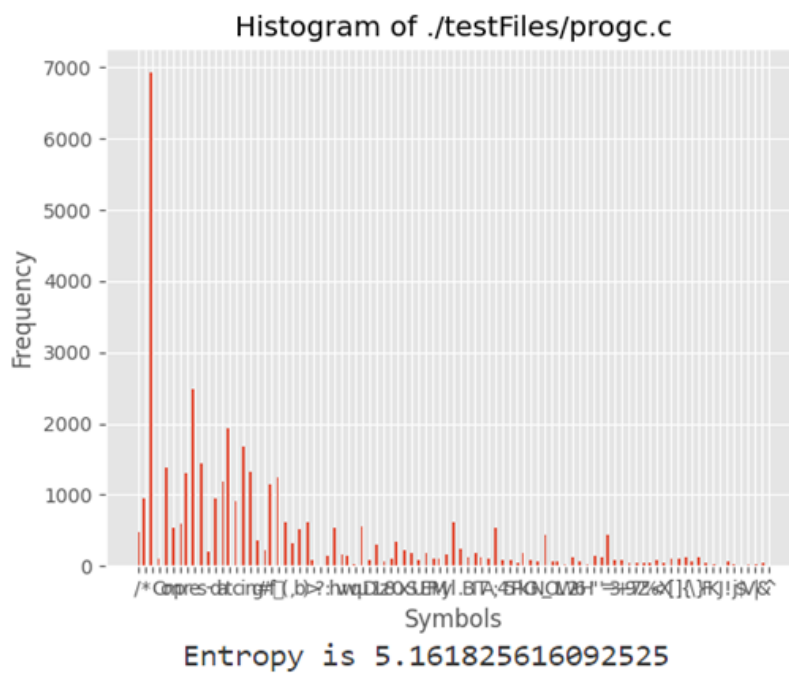


figura 8 - Histograma e Entropia do Progc.c

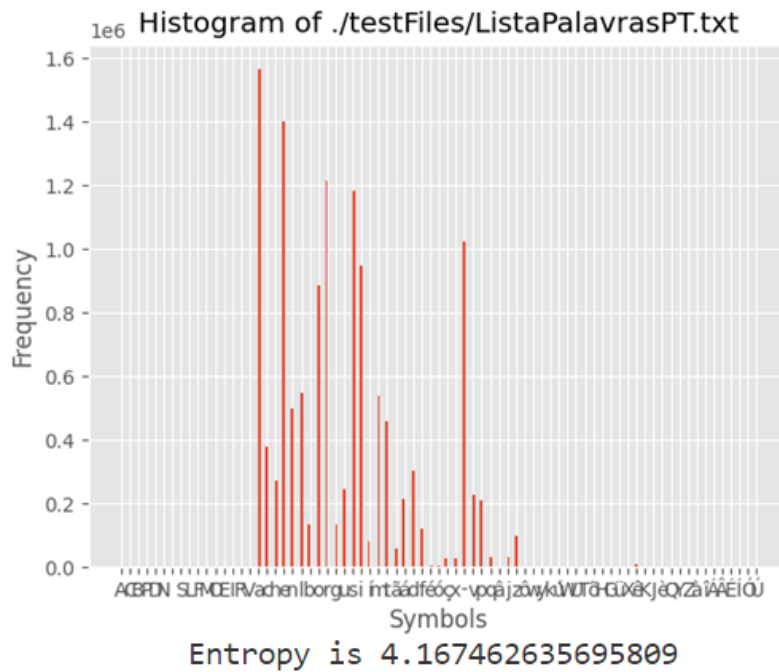


figura 10 - Histograma e Entropia do ListaPalavrasPT.txt

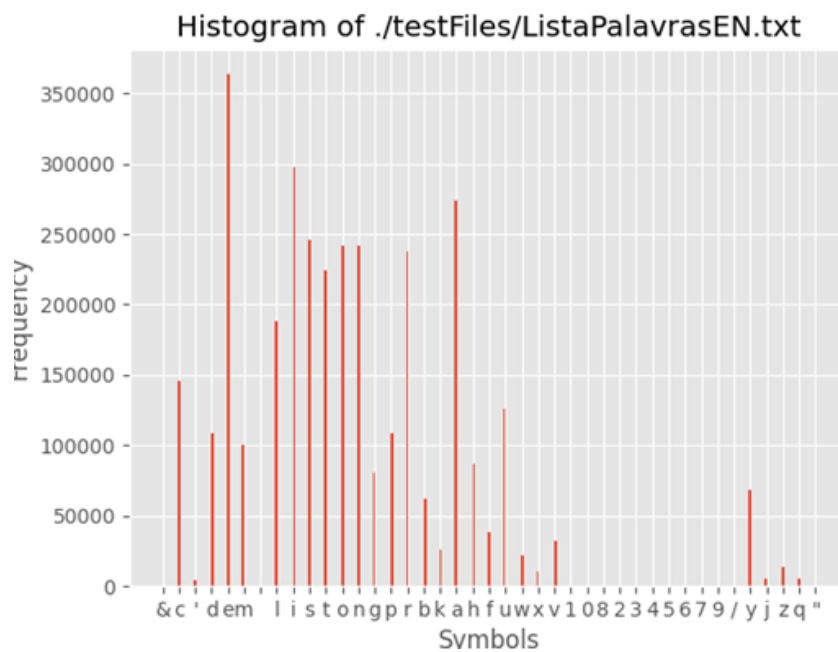


figura 9 - Histograma e Entropia do ListaPalavrasEN.txt

3.1.2 Alínea 2

3.1.2.1 Alínea (a) – *histogram.py*

Nesta alínea, foi implementado um código capaz de processar os ficheiros presentes no conjunto TestFilesCD.zip. O código realiza a leitura de cada ficheiro e, em seguida, gera o histograma dos símbolos presentes no mesmo. Isto permite visualizar a distribuição dos símbolos e identificar os símbolos mais frequentes e menos frequentes no conjunto de dados.

Além disso, o código calcula o valor da informação própria de cada símbolo presente no ficheiro. A informação própria é uma medida que representa a quantidade de informação que um símbolo fornece quando ocorre. Quanto menor a probabilidade de ocorrência de um símbolo, maior será a informação própria associada a ele.

Por fim, o código também tem a capacidade de calcular, através das fórmulas aprendidas em aula, o valor da entropia do ficheiro. A entropia é uma medida que indica a quantidade média de informação necessária para representar cada símbolo do ficheiro. Quanto maior a entropia, maior será a aleatoriedade ou a incerteza associada aos símbolos presentes no ficheiro.

Nesta alínea, foi utilizada a biblioteca matplotlib.pyplot para mostrar o histograma dos símbolos presentes nos ficheiros de teste fornecidos. Essa biblioteca é amplamente utilizada para criar visualizações gráficas em Python, incluindo gráficos de barras, como é o caso do histograma.

3.1.3 Alínea 3

3.1.3.1 Alínea (a) - *GenericSource.py*

A implementação consiste em criar uma fonte de símbolos genérica que gera ficheiros com N símbolos, onde N é definido pelo utilizador. O utilizador deve especificar a função massa de probabilidade (FMP) do alfabeto de M símbolos, bem como a lista de símbolos correspondente.

A função começa por ajustar a FMP de forma que a soma das probabilidades seja igual a M, para garantir que a lista de símbolos gerados tem um tamanho exato de M. Em seguida, é criado um ficheiro de texto vazio para armazenar a sequência de símbolos gerada.

O próximo passo é gerar aleatoriamente os símbolos da sequência de acordo com a FMP definida, enquanto a sequência não estiver completa. Para gerar cada símbolo, é gerado um número aleatório entre 0 e o comprimento da lista de probabilidades $p_x - 1$, e o símbolo correspondente a essa probabilidade é adicionado à sequência. A probabilidade desse símbolo é

então reduzida em 1, e se a probabilidade for agora 0, o símbolo e a sua probabilidade são removidos das listas.

Finalmente, a sequência de símbolos é escrita no ficheiro de texto e o ficheiro é fechado. O resultado é um ficheiro de texto contendo uma sequência de M símbolos, gerados aleatoriamente de acordo com a FMP definida.

3.1.3.2 Alínea (b) - *SequenceGenerator.py*

A implementação consiste em criar uma fonte de símbolos genérica que gera sequências de diferentes dimensões N, para valores de N especificados pelo utilizador. Para cada valor de N, é gerada uma sequência aleatória de símbolos com base na função massa de probabilidade (FMP) especificada pelo utilizador.

A função genérica de fonte de símbolos é alterada para garantir que a soma das probabilidades de todas as possíveis saídas seja igual a M, a dimensão total da sequência gerada. Além disso, a função de escolha de símbolos é alterada para seleccionar os símbolos com base nas suas probabilidades, em vez de seleccionar aleatoriamente de forma uniforme.

Para estimar a entropia da sequência gerada, são utilizadas duas funções: a função de entropia, que calcula a entropia teórica com base na FMP especificada, e a função de entropia estimada, que calcula a entropia da sequência gerada com base nas frequências observadas dos símbolos na sequência.

Finalmente, a função principal itera sobre diferentes dimensões N especificadas pelo utilizador e imprime a entropia teórica e estimada para cada dimensão N. Os resultados são comentados com base na comparação entre a entropia teórica e a entropia estimada para diferentes valores de M e N.

3.1.3.2 Alínea (c) - *PasswordGenerator.py*

Este exercício consiste na implementação da fonte de símbolos, para realizar um gerador de palavras-passe robustas, com dimensão entre 8 e 12 caracteres.

Esta implementação utiliza a função `genericSource` (a) para gerar palavras-passe robustas com dimensão entre 8 e 12 caracteres. A função `generatePassword` define a função de probabilidade de massa `px` e o alfabeto de símbolos `symbols` a serem utilizados na geração das palavras-passe. Em seguida, é seleccionado aleatoriamente um tamanho `length` para a palavra-passe, entre 8 e 12

caracteres. Finalmente, a função `genericSource` é chamada com os argumentos `px`, `length` e `symbols` para gerar uma palavra-passe aleatória.

A função `main` chama a função `generatePassword` cinco vezes e imprime cada palavra-passe gerada. A saída do programa apresenta cinco palavras-passe aleatórias geradas com base na probabilidade de massa `px` e no alfabeto de símbolos `symbols`.

3.1.4 Alínea 4

3.1.4.1 Alínea (a) – *VernamCypher.py*

A função `makeVernamCypher` recebe como entrada duas sequências de caracteres com o mesmo tamanho, `plainText` e `theKey`. Essa função realiza a cifra de Vernam, que consiste em aplicar a operação XOR bit a bit de todos os caracteres das duas sequências para obter uma nova sequência de caracteres criptografados, `cipherText`. Para cada posição `i` na sequência `plainText`, a função realiza a operação XOR dos caracteres correspondentes em `plainText` e `theKey`. O resultado é convertido em um caractere ASCII correspondente e adicionado à sequência `cipherText`. Ao final, a função retorna uma sequência criptografada `cipherText`.

No caso específico da implementação fornecida, a função verifica se o tamanho de `plainText` e `theKey` são iguais. Se não forem, uma exceção é levantada indicando que os dois argumentos devem ter o mesmo tamanho. Caso contrário, a função percorre cada caractere em `plainText` e realiza a operação XOR bit a bit com o caractere correspondente em `theKey`. O resultado é adicionado à sequência `cipherText`, que é retornada ao final.

Para demonstrar o funcionamento da implementação, é fornecido um exemplo de cifragem e decifragem da sequência `abcabcd` com a chave constante `3333333`. A cifra produzida é `ããããã\`. A decifragem dessa cifra com a mesma chave resulta na sequência original `abcabcd`.

3.1.4.2 Alínea (b) – *AliceCypher.py*

O programa lê o arquivo `"alice29.txt"` contendo o texto em claro que será cifrado com a cifra de Vernam. Em seguida, é definido um tamanho de bin para a criação dos histogramas, que serão utilizados para análise da distribuição dos caracteres no texto em claro e no texto cifrado.

A função `makeVernamCypher` é importada a partir do módulo `VernamCypher` para realizar a cifragem do texto em claro. É definida uma chave para ser utilizada na cifra de Vernam, que pode ser uma sequência de caracteres aleatória ou uma chave constante.

O texto em claro é cifrado utilizando a função `makeVernamCypher`, que retorna uma sequência cifrada `ciphertext`. Em seguida, são criados dois histogramas: um para o texto em claro e outro para o texto cifrado. Cada histograma é criado a partir da contagem de ocorrências de cada valor ASCII encontrado no texto correspondente.

Os histogramas são criados com a biblioteca `Matplotlib`. Cada valor ASCII encontrado no texto é agrupado em um intervalo de `bin_size` valores, e o número de ocorrências em cada intervalo é contado e exibido em um gráfico de barras.

Ao final, os histogramas são exibidos em janelas separadas, cada uma com um título indicando se é referente ao texto em claro ou ao texto cifrado. Essa análise pode ser útil para entender o impacto da cifra de Vernam na distribuição dos caracteres do texto.

É importante notar que, além dos histogramas, também são calculadas as entropias do texto em claro e do texto cifrado. A entropia é uma medida da incerteza associada a uma distribuição de probabilidade. Ela é utilizada para avaliar o grau de aleatoriedade de um conjunto de dados. Comparar as entropias do texto em claro e do texto cifrado pode ajudar a avaliar o nível de segurança da cifra utilizada.

3.1.5 Alínea 5

3.1.5.1 Alínea (a) – *Binary Symmetric Channel*

A função `BSC()` dado uma sequência binária e um valor de BER, gera uma nova sequência binária em função da BER introduzida. Esta função tem como objetivo simular a passagem de uma trama de bits por um canal ruidoso de forma a haver troca de bits na mensagem e como consequência uma mensagem diferente da original. A função começa por converter a Sequencia binária recebida como parâmetro e numa *array* de inteiros, isto é, zeros e uns. Em seguida é feito um ciclo por cada bit irá ser gerado um valor esse valor se for menor que a BER, isto é, se a BER tiver um valor de 0.25, 25% dos bits poderão ser trocados, todos os valores desde 0.00 a 0.25 tem a mesma chance de ocorrer e são 25% das hipóteses totais, ou seja, a cada bit da sequência binária e aplicada a probabilidade da BER e trocado o bit se o valor gerado for menor que esta. No fim é transformado o *array* de inteiros numa *string* de zeros e uns.

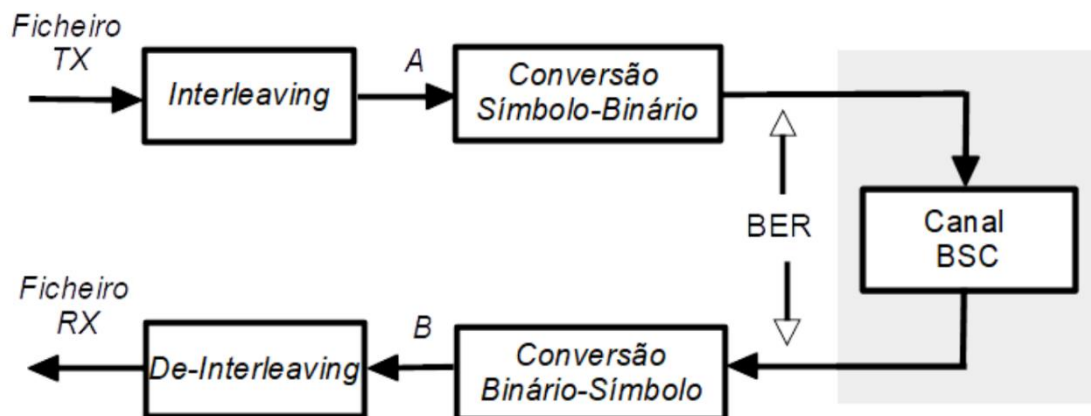
3.1.5.2 Alínea (b) – Técnica de entrelaçamento (interleaving)

A função `interleaving()` gera uma matriz de caracteres com base numa mensagem e dimensões da matriz dadas. Esta função recebe como parâmetro as dimensões da matriz desejada, isto é, número de colunas, número de linhas e recebe uma mensagem, sequência de caracteres a colocar na matriz. A função começa por criar uma matriz das dimensões fornecidas vazia, e com base

em 2 ciclos, um ciclo que percorre as linhas e outro que percorre as colunas vai introduzindo os caracteres da mensagem na matriz, preenchendo por linha.

A função `deinterleaving()` gera uma mensagem dado uma matriz de caracteres. Esta função começa por calcular o número de linhas e número de colunas da matriz. Em seguida é feito 2 ciclos, um que percorre as linhas e outro que percorre as colunas ao mesmo tempo que é introduzido numa sequência de caracteres, os caracteres da matriz.

3.1.5.3 Alínea (b) – técnica de entrelaçamento em conjunto com o modelo de canal BSC



Na figura acima está representada a aplicação da técnica de entrelaçamento proposta pelo enunciado, este objetivo foi alcançado e implementado no ficheiro “`interleaving_bsc_deinterleaving.py`”. Este ficheiro contém uma função `main()` onde é executado as funções `interleaving()`, `matrix_to_binary()`, `BSC()`, `binary_to_matrix` e `deinterleaving()`. A função `main()` começa por chamar a função `interleaving()` passando como parâmetro a dimensão da matriz e mensagem TX, esta retorna uma matriz que será convertida para binário. Em seguida, é chamada a função `BSC()` que dado uma BER, definida na `main()`, simula a passagem da mensagem enviada por um canal ruidoso, ou seja, altera os bits da mensagem. Em seguida é feito uma chamada á função `binary_to_matrix()` que dada uma sequência binária converte numa matriz e de seguida é chamada a função `deinterleaving` para apresentar a mensagem RX, a mensagem recebida. Por fim é comparada a mensagem em

binário produzida após a passagem pelo Binary Symmetric Channel com a mensagem antes da passagem pelo canal, e obtemos o número de bits trocados, com o número de bits trocados e com o número total de bits podemos concluir a BER real, após a passagem pelo canal, usando a fórmula: $BER = \frac{\#bits\ errados}{\#bits\ total}$.

3.2 Expressões matemáticas

$$H(X) = E[-\log_2 p(x_i)] \quad I(X) = -\log_2(p(x))$$

$$P(X) = 2^{-I(X)} \quad P(X_1) + P(X_2) + \dots + P(X_i) = 1$$

4. Avaliação Experimental

4.1 Análise Experimental

4.1.1 Alínea 4

4.1.1.1 (a) – *count_ones.c* e *count_zeros.c*

No teste das funções ‘count_ones’ e ‘count_zeros’ vamos chamando a função e comparando com um valor esperado.

4.1.1.2 (b) – *print_bits.c*

No teste do ‘print_bits’ nós redirecionamos o output da consola para um ficheiro local para o podermos examinar e copiamos o conteúdo deste para um buffer, como esperamos no máximo 32 bits imprimidos, o buffer de 100 é suficiente. Depois simplesmente comparamos o conteúdo do buffer com o valor esperado.

4.1.1.3 (c) – *most_frequent_symbol.c*

No teste da função ‘most_frequent_symbol’ chamamos a função com um ficheiro que contém a letra ‘a’ muitas vezes e comparamos o resultado dela com este valor esperado.

4.1.1.4 (d) – *negative_file.c*

No teste da função ‘negative_file’, também redirecionamos o output da consola para um ficheiro local. Esperamos para um caracter 1, o output 206 pois o código ASCII mais alto (255) – o código ASCII do caracter a inverter = $255 - 49 = 206$.

4.1.2 Alínea 5

4.1.2.1 (a) – *geometric_progression_test.py*

O código apresentado realiza um teste para verificar se a função "geometric_prog" implementada no arquivo "geometric_progression.py" está a calcular corretamente os termos de uma progressão geométrica com os parâmetros especificados.

O teste foi criado com o objetivo de verificar se a implementação da função "geometric_prog" está a produzir uma sequência de números que segue a fórmula matemática para uma progressão

geométrica, que é $a_n = a_1 * r^{(n-1)}$, onde a_n é o n -ésimo termo da sequência, a_1 é o primeiro termo, r é a razão comum entre os termos e n é o número de termos.

O teste consiste em chamar a função "geometric_prog" com os argumentos $u=3$, $r=2$ e $n=10$, e em seguida comparar cada um dos termos gerados pela função com a fórmula matemática para a progressão geométrica. Se todos os termos gerados pela função estiverem corretos, o teste é considerado bem-sucedido e uma mensagem "Test1 Succeeded!" é exibida na saída. Caso contrário, o teste é considerado como falhado e uma mensagem "Test1 Failed!" é exibida.

4.1.2.2 (b) - EuclidesAlgorithm_test.py

O código apresentado realiza um teste para verificar se a função "MDC" implementada no arquivo "EuclidesAlgorithm.py", que tem como objetivo calcular corretamente o máximo divisor comum (MDC) entre dois números.

O teste foi criado com o objetivo de verificar se a implementação da função "MDC" produz resultados consistentes. Para isso, a função "Test1" foi criada para testar o MDC entre os números 348 e 156.

O teste consiste em chamar a função "MDC" com os argumentos 348 e 156, e em seguida comparar o resultado retornado pela função com o resultado da função "gcd" do módulo "math". Se os resultados forem iguais, o teste é considerado bem-sucedido e uma mensagem "Test1 Succeeded!" é exibida na saída. Caso contrário, o teste é considerado como falhado e uma mensagem "Test1 Failed!" é exibida.

4.1.2.3 (c) - most_and_least_frequent_symbols.py

O código apresentado é um conjunto de testes para a função `most_and_least_frequent_symbols`. Esta função recebe um arquivo de texto como entrada e retorna o símbolo mais frequente e o menos frequente no arquivo juntamente com as suas contagens.

Foram realizados três testes diferentes usando arquivos de texto diferentes para validar a função `most_and_least_frequent_symbols`. Cada teste verifica se a função retorna o símbolo mais frequente e o menos frequente e as suas contagens corretas.

Se a função retornar os valores esperados, o teste é considerado um sucesso e a mensagem "Test Succeeded!" é exibida no console. Caso contrário, o teste falha e a mensagem "Test Failed!" é exibida.

4.1.2.4 (d) - *histogram.py*

O código apresenta uma função `histogram` que recebe como argumento um nome de arquivo. O objetivo dessa função é calcular a entropia e gerar um histograma com a frequência de ocorrência dos símbolos presentes no arquivo.

O teste `test1` verifica se a entropia calculada pela função `histogram` para o arquivo `test_file_5d.txt` está correta. Caso a entropia calculada seja igual a 1.8394910703001344, o teste é considerado bem-sucedido. Caso contrário, é considerado como falhado.

5. Conclusões

Com base nos exercícios propostos, podemos concluir que a análise e implementação de fontes de símbolos, cifras de segurança e modelos de canal binário simétrico são tópicos fundamentais na área da segurança e comunicações. Sendo que, a resolução dos mesmo foi executada com sucesso e eficiência, assim como os testes associados a eles, respetivamente. A análise de fontes de símbolos permite obter informação valiosa acerca da entropia e distribuição dos símbolos em diferentes ficheiros, o que é relevante para a escolha de técnicas de compressão, encriptação e deteção de anomalias.

A implementação de fontes de símbolos genéricas, assim como a geração de palavras-passe robustas, é uma prática importante para garantir a segurança de sistemas e comunicações. Além disso, a cifra de Vernam apresenta-se como uma técnica de cifragem robusta, cuja implementação permite garantir a confidencialidade de informação sensível.

Por fim, a utilização de técnicas de entrelaçamento em sistemas de comunicação é uma prática importante para mitigar os efeitos de interferências e erros, como aqueles introduzidos pelo modelo de canal binário simétrico. A comparação entre as sequências nos pontos A e B, assim como a análise dos valores de BER, permitem avaliar a eficácia da técnica de entrelaçamento na redução de erros e na melhoria da qualidade da transmissão.

Em suma, os exercícios propostos permitiram um aprofundamento no conhecimento sobre fontes de símbolos, implementação de algoritmos em Python e C, e técnicas de transmissão de dados.

Referências

- [1] A Byte of Python - Meta Disciplina - Comunicação Digital - LEIC - 2223SV
- [2] Slides - Meta Disciplina - Comunicação Digital - LEIC - 2223SV